

条件运算符满足右结合律，意味着运算对象（一般）按照从右向左的顺序组合。因此上面的代码中，靠右边的条件运算（比较成绩是否小于 60）构成了靠左边的条件运算的分支。



随着条件运算嵌套层数的增加，代码的可读性急剧下降。因此，条件运算的嵌套最好别超过两到三层。

在输出表达式中使用条件运算符

条件运算符的优先级非常低，因此当一条长表达式中嵌套了条件运算符表达式时，通常需要在它两端加上括号。例如，有时需要根据条件值输出两个对象中的一个，如果写这条语句时没把括号写全就有可能产生意想不到的结果：

152

```
cout << ((grade < 60) ? "fail" : "pass"); // 输出 pass 或者 fail
cout << (grade < 60) ? "fail" : "pass"; // 输出 1 或者 0!
cout << grade < 60 ? "fail" : "pass"; // 错误：试图比较 cout 和 60
```

在第二条表达式中，grade 和 60 的比较结果是 << 运算符的运算对象，因此如果 grade < 60 为真输出 1，否则输出 0。<< 运算符的返回值是 cout，接下来 cout 作为条件运算符的条件。也就是说，第二条表达式等价于

```
cout << (grade < 60); // 输出 1 或者 0
cout ? "fail" : "pass"; // 根据 cout 的值是 true 还是 false 产生对应的字面值
```

因为第三条表达式等价于下面的语句，所以它是错误的：

```
cout << grade; // 小于运算符的优先级低于移位运算符，所以先输出 grade
cout < 60 ? "fail" : "pass"; // 然后比较 cout 和 60!
```

4.7 节练习

练习 4.21: 编写一段程序，使用条件运算符从 `vector<int>` 中找到哪些元素的值是奇数，然后将这些奇数值翻倍。

练习 4.22: 本节的示例程序将成绩划分成 high pass、pass 和 fail 三种，扩展该程序使其进一步将 60 分到 75 分之间的成绩设定为 low pass。要求程序包含两个版本：一个版本只使用条件运算符；另外一个版本使用 1 个或多个 if 语句。哪个版本的程序更容易理解呢？为什么？

练习 4.23: 因为运算符的优先级问题，下面这条表达式无法通过编译。根据 4.12 节中的表（第 147 页）指出它的问题在哪里？应该如何修改？

```
string s = "word";
string p1 = s + s[s.size() - 1] == 's' ? "" : "s";
```

练习 4.24: 本节的示例程序将成绩划分成 high pass、pass 和 fail 三种，它的依据是条件运算符满足右结合律。假如条件运算符满足的是左结合律，求值过程将是怎样的？

4.8 位运算符

位运算符作用于整数类型的运算对象，并把运算对象看成是二进制位的集合。位运算符提供检查和设置二进制位的功能，如 17.2 节（第 640 页）将要介绍的，一种名为 `bitset`

的标准库类型也可以表示任意大小的二进制位集合，所以位运算符同样能用于 `bitset` 类型。

153

表 4.3: 位运算符（左结合律）

运算符	功能	用法
<code>~</code>	位求反	<code>~ expr</code>
<code><<</code>	左移	<code>expr1 << expr2</code>
<code>>></code>	右移	<code>expr1 >> expr2</code>
<code>&</code>	位与	<code>expr & expr</code>
<code>^</code>	位异或	<code>expr ^ expr</code>
<code> </code>	位或	<code>expr expr</code>

一般来说，如果运算对象是“小整型”，则它的值会被自动提升（参见 4.11.1 节，第 142 页）成较大的整数类型。运算对象可以是带符号的，也可以是无符号的。如果运算对象是带符号的且它的值为负，那么位运算符如何处理运算对象的“符号位”依赖于机器。而且，此时的左移操作可能会改变符号位的值，因此是一种未定义的行为。



关于符号位如何处理没有明确的规定，所以强烈建议仅将位运算符用于处理无符号类型。

移位运算符

之前在处理输入和输出操作时，我们已经使用过标准 IO 库定义的 `<<` 运算符和 `>>` 运算符的重载版本。这两种运算符的内置含义是对其运算对象执行基于二进制位的移动操作，首先令左侧运算对象的内容按照右侧运算对象的要求移动指定位数，然后将经过移动的（可能还进行了提升）左侧运算对象的拷贝作为求值结果。其中，右侧的运算对象一定不能为负，而且值必须严格小于结果的位数，否则就会产生未定义的行为。二进制位或者向左移（`<<`）或者向右移（`>>`），移出边界之外的位就被舍弃掉了：

在下面的图例中右侧为最低位并且假定 `char` 占 8 位、`int` 占 32 位
// 0233 是八进制的字面值（参见 2.1.3 节，第 35 页）

```
unsigned char bits = 0233;
bits << 8 // bits 提升成 int 类型，然后向左移动 8 位
bits << 31 // 向左移动 31 位，左边超出边界的位丢弃了
bits >> 3 // 向右移动 3 位，最右边的 3 位丢弃了
```

1 0 0 1 1 0 1 1

0 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0 | 1 0 0 1 1 0 1 1 | 0 0 0 0 0 0 0 0

1 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0 | 0 0 0 1 0 0 1 1

左移运算符（`<<`）在右侧插入值为 0 的二进制位。右移运算符（`>>`）的行为则依赖于其左侧运算对象的类型：如果该运算对象是无符号类型，在左侧插入值为 0 的二进制位；如果该运算对象是带符号类型，在左侧插入符号位的副本或值为 0 的二进制位，如何选择要视具体环境而定。

154

位求反运算符

位求反运算符（`~`）将运算对象逐位求反后生成一个新值，将 1 置为 0、将 0 置为 1：

```

unsigned char bits = 0227;    1 0 0 1 0 1 1 1
~bits
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 0 1 0 0 0

```

char 类型的运算对象首先提升成 int 类型，提升时运算对象原来的位保持不变，往高位（high order position）添加 0 即可。因此在本例中，首先将 bits 提升成 int 类型，增加 24 个高位 0，随后将提升后的值逐位求反。

位与、位或、位异或运算符

与（&）、或（|）、异或（^）运算符在两个运算对象上逐位执行相应的逻辑操作：

```

unsigned char b1 = 0145;    0 1 1 0 0 1 0 1
unsigned char b2 = 0257;    1 0 1 0 1 1 1 1
b1 & b2    24 个高阶位都是0    0 0 1 0 0 1 0 1
b1 | b2    24 个高阶位都是0    1 1 1 0 1 1 1 1
b1 ^ b2    24 个高阶位都是0    1 1 0 0 1 0 1 0

```

对于位与运算符（&）来说，如果两个运算对象的对应位置都是 1 则运算结果中该位为 1，否则为 0。对于位或运算符（|）来说，如果两个运算对象的对应位置至少有一个为 1 则运算结果中该位为 1，否则为 0。对于位异或运算符（^）来说，如果两个运算对象的对应位置有且只有一个为 1 则运算结果中该位为 1，否则为 0。



有一种常见的错误是把位运算符和逻辑运算符（参见 4.3 节，第 126 页）搞混了，比如位与（&）和逻辑与（&&）、位或（|）和逻辑或（||）、位求反（~）和逻辑非（!）。

使用位运算符

我们举一个使用位运算符的例子：假设班级中有 30 个学生，老师每周都会对学生进行一次小测验，测验的结果只有通过和不通过两种。为了更好地追踪测验的结果，我们用一个二进制位代表某个学生在一次测验中是否通过，显然全班的测验结果可以用一个无符号整数来表示：

```
unsigned long quiz1 = 0;    // 我们把这个值当成是位的集合来使用
```

定义 quiz1 的类型是 unsigned long，这样，quiz1 在任何机器上都将至少拥有 32 位；给 quiz1 赋一个明确的初始值，使得它的每一位在开始时都有统一且固定的值。

155

教师必须有权设置并检查每一个二进制位。例如，我们需要对序号为 27 的学生对应的位进行设置，以表示他通过了测验。为了达到这一目的，首先创建一个值，该值只有第 27 位是 1 其他位都是 0，然后将这个值与 quiz1 进行位或运算，这样就能强行将 quiz1 的第 27 位设置为 1，其他位都保持不变。

为了实现本例的目的，我们将 quiz1 的低阶位赋值为 0、下一位赋值为 1，以此类推，最后统计 quiz1 各个位的情况。

使用左移运算符和一个 unsigned long 类型的整数字面值 1（参见 2.1.3 节，第 35 页）就能得到一个表示学生 27 通过了测验的数值：

```
1UL << 27    // 生成一个值，该值只有第 27 位为 1
```

1UL 的低阶位上有一个 1, 除此之外(至少)还有 31 个值为 0 的位。之所以使用 unsigned long 类型, 是因为 int 类型只能确保占用 16 位, 而我们至少需要 27 位。上面这条表达式通过在值为 1 的那个二进制位后面添加 0, 使得它向左移动了 27 位。

接下来将所得的值与 quiz1 进行位或运算。为了同时更新 quiz1 的值, 使用一条复合赋值语句(参见 4.4 节, 第 130 页):

```
quiz1 |= 1UL << 27;           // 表示学生 27 通过了测验
```

|= 运算符的工作原理和 += 非常相似, 它等价于

```
quiz1 = quiz1 | 1UL << 27;    // 等价于 quiz1 |= 1UL << 27;
```

假定教师在重新核对测验结果时发现学生 27 实际上并没有通过测验, 他必须要把第 27 位的值置为 0。此时我们需要使用一个特殊的整数, 它的第 27 位是 0、其他所有位都是 1。将这个值与 quiz1 进行位与运算就能实现目的了:

```
quiz1 &= ~(1UL << 27);       // 学生 27 没有通过测验
```

通过将之前的值按位求反得到一个新值, 除了第 27 位外都是 1, 只有第 27 位的值是 0。随后将该值与 quiz1 进行位与运算, 所得结果除了第 27 位外都保持不变。

最后, 我们试图检查学生 27 测验的情况到底怎么样:

```
bool status = quiz1 & (1UL << 27); // 学生 27 是否通过了测验?
```

我们将 quiz1 和一个只有第 27 位是 1 的值按位求与, 如果 quiz1 的第 27 位是 1, 计算的结果就是非 0 (真); 否则结果是 0。



移位运算符(又叫 IO 运算符)满足左结合律

尽管很多程序员从未直接用过位运算符, 但是几乎所有人都用过它们的重载版本来进行 IO 操作。重载运算符的优先级和结合律都与它的内置版本一样, 因此即使程序员用不到移位运算符的内置含义, 也仍然有必要理解其优先级和结合律。

因为移位运算符满足左结合律, 所以表达式

156

```
cout << "hi" << " there" << endl;
```

的执行过程实际上等同于

```
( (cout << "hi") << " there" ) << endl;
```

在这条语句中, 运算对象 "hi" 和第一个 << 组合在一起, 它的结果和第二个 << 组合在一起, 接下来的结果再和第三个 << 组合在一起。

移位运算符的优先级不高不低, 介于中间: 比算术运算符的优先级低, 但比关系运算符、赋值运算符和条件运算符的优先级高。因此在一次使用多个运算符时, 有必要在适当的地方加上括号使其满足我们的要求。

```
cout << 42 + 10;           // 正确: + 的优先级更高, 因此输出求和结果
cout << (10 < 42);         // 正确: 括号使运算对象按照我们的期望组合在一起, 输出 1
cout << 10 < 42;           // 错误: 试图比较 cout 和 42!
```

最后一个 cout 的含义其实是

```
(cout << 10) < 42;
```

也就是“把数字 10 写到 cout, 然后将结果(即 cout)与 42 进行比较”。

4.8 节练习

练习 4.25: 如果一台机器上 `int` 占 32 位、`char` 占 8 位，用的是 Latin-1 字符集，其中字符 'q' 的二进制形式是 01110001，那么表达式 `~'q' << 6` 的值是什么？

练习 4.26: 在本节关于测验成绩的例子中，如果使用 `unsigned int` 作为 `quiz1` 的类型会发生什么情况？

练习 4.27: 下列表达式的结果是什么？

```
unsigned long ul1 = 3, ul2 = 7;
```

(a) `ul1 & ul2`

(b) `ul1 | ul2`

(c) `ul1 && ul2`

(d) `ul1 || ul2`

4.9 sizeof 运算符

`sizeof` 运算符返回一条表达式或一个类型名字所占的字节数。`sizeof` 运算符满足右结合律，其所得的值是一个 `size_t` 类型（参见 3.5.2 节，第 103 页）的常量表达式（参见 2.4.4 节，第 58 页）。运算符的运算对象有两种形式：

```
sizeof (type)
sizeof expr
```

在第二种形式中，`sizeof` 返回的是表达式结果类型的大小。与众不同的一点是，`sizeof` 并不实际计算其运算对象的值：

```
Sales_data data, *p;
sizeof(Sales_data);    // 存储 Sales_data 类型的对象所占的空间大小
sizeof data;           // data 的类型的大小，即 sizeof(Sales_data)
sizeof p;              // 指针所占的空间大小
sizeof *p;             // p 所指类型的空间大小，即 sizeof(Sales_data)
sizeof data.revenue;   // Sales_data 的 revenue 成员对应类型的大小
sizeof Sales_data::revenue; // 另一种获取 revenue 大小的方式
```

157

这些例子中最有趣的一个是 `sizeof *p`。首先，因为 `sizeof` 满足右结合律并且与 `*` 运算符的优先级一样，所以表达式按照从右向左的顺序组合。也就是说，它等价于 `sizeof(*p)`。其次，因为 `sizeof` 不会实际求运算对象的值，所以即使 `p` 是一个无效（即未初始化）的指针（参见 2.3.2 节，第 47 页）也不会有什么影响。在 `sizeof` 的运算对象中解引用一个无效指针仍然是一种安全的行为，因为指针实际上并没有被真正使用。`sizeof` 不需要真的解引用指针也能知道它所指对象的类型。

C++11 新标准允许我们使用作用域运算符来获取类成员的大小。通常情况下只有通过类的对象才能访问到类的成员，但是 `sizeof` 运算符无须我们提供一个具体的对象，因为要想知道类成员的大小无须真的获取该成员。

C++
11

`sizeof` 运算符的结果部分地依赖于其作用的类型：

- 对 `char` 或者类型为 `char` 的表达式执行 `sizeof` 运算，结果得 1。
- 对引用类型执行 `sizeof` 运算得到被引用对象所占空间的大小。
- 对指针执行 `sizeof` 运算得到指针本身所占空间的大小。
- 对解引用指针执行 `sizeof` 运算得到指针指向的对象所占空间的大小，指针不需有效。